Software Quality Assurance, Software Metrics, and Software Maintenance

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Explain software quality and distinguish quality assurance from testing.

• Describe quality planning, reviews, audits, standards, and defect prevention.

• Interpret basic software metrics and understand their limitations.

• Explain the major types of software maintenance and why maintenance is necessary.

• Apply quality and maintenance ideas to a College Event Management System (CEMS).

Running case study: CEMS supports event discovery, student registration, organizer administration, and attendance tracking.

1. What Is Software Quality?

Software quality is the degree to which a software product satisfies stated requirements, relevant standards, and the needs of its users in its intended context. Quality is not only whether a program runs: it also includes whether it is reliable, secure, usable, maintainable, efficient, and appropriate for its purpose.

For CEMS, quality means students can register correctly, organizers can manage events, personal information is protected, pages respond acceptably, and the team can safely change the application when college rules evolve.

Quality depends on context. A prototype built to test an idea has different quality expectations from a production application handling sensitive student information.

2. Quality Assurance vs. Quality Control vs. Testing

Term	Main focus	Typical examples
Quality Assurance (QA)	Improving the process so defects are less likely to be introduced	Defined development process, coding guidelines, training, audits, review practices
Quality Control (QC)	Evaluating work products to identify whether quality criteria are met	Inspecting a requirements document, reviewing a build, checking deliverables
Testing	Executing software or otherwise exercising it to reveal defects and evaluate behavior	Running registration test cases, API tests, load tests

Testing is an important quality activity, but QA is broader than testing. QA is primarily process-oriented and preventive; QC and testing evaluate product quality. In practice, teams combine all of these activities.

3. Quality Assurance Activities

3.1 Quality planning

A quality plan describes relevant quality goals, standards, responsibilities, review points, testing expectations, tools, and acceptance criteria. Goals should be specific enough to evaluate.

3.2 Reviews and inspections

A review brings people together to examine a work product such as an SRS, UML diagram, code change, test plan, or user guide. Reviewers can identify ambiguity, missing cases, inconsistencies, and risks before those issues become expensive to fix.

3.3 Standards and guidelines

Teams may agree on naming conventions, coding standards, documentation templates, secure-development practices, and version-control rules. Consistent practices make work easier to review and maintain.

3.4 Audits and process improvement

An audit checks whether required processes or standards are being followed. Process improvement uses evidence—such as recurring defects, review findings, or missed acceptance criteria—to adjust how work is done.

3.5 Defect prevention

• Clarify requirements and acceptance criteria before implementation.

• Use peer reviews for high-risk changes.

• Automate repeatable checks where practical.

• Track recurring defect patterns and improve the process that allowed them.

• Keep responsibilities, approvals, and change history visible.

4. Quality Attributes and Practical Measures

Attribute	Meaning	Possible evidence
Reliability	Ability to operate correctly over time and conditions	Failure rate, incident records, successful transaction rate
Usability	How effectively users can complete tasks	Task completion, user feedback, observed usability issues
Performance	Responsiveness and resource use	Response-time percentiles, throughput, CPU/memory use
Security	Protection against unauthorized access and misuse	Security test results, access-control checks, resolved vulnerabilities
Maintainability	Ease of understanding, testing, and changing software	Change lead time, defect rate after changes, review findings
Availability	How consistently a service is usable	Uptime percentage and incident duration

A measure is useful only when its definition, collection method, context, and limitations are understood. One number rarely tells the whole quality story.

5. Software Metrics

A software metric is a quantitative measure related to a software product, process, or project. Metrics help teams identify trends, estimate work, understand risk, and make improvement decisions. They should support discussion and learning—not replace judgment.

5.1 Product metrics

Product metrics describe properties of the software or its artefacts.

• Defect density: number of recorded defects divided by a defined size measure, such as thousands of lines of code. Comparisons are meaningful only when counting rules and size measures are consistent.

• Cyclomatic complexity: an estimate of the number of linearly independent paths through a program's control flow; higher values can signal code that may be harder to test or maintain.

• Code coverage: the proportion of selected code elements exercised by tests. High coverage does not guarantee that assertions are meaningful or that requirements are satisfied.

5.2 Process metrics

Process metrics describe how work is performed.

• Review effectiveness: issues identified during review, interpreted alongside escaped defects and review effort.

• Defect removal trend: when and where defects are detected and resolved.

• Rework rate: effort spent correcting or redoing previously completed work, using a consistent definition.

5.3 Project metrics

Project metrics support planning and monitoring.

• Schedule variance: difference between planned and actual timing, with the project's calculation convention stated.

• Effort variance: difference between estimated and actual effort.

• Sprint throughput or velocity: team-specific measures of completed work; these are planning aids, not reliable cross-team productivity rankings.

6. Worked Metric Examples

Example A: Defect density

Suppose a release contains 12 recorded defects and the measured size is 6 KLOC (six thousand lines of code).

> Defect density = 12 ÷ 6 = **2 defects per KLOC**.

This value does not by itself prove the release is good or bad. The severity of defects, test scope, software type, and counting rules all matter.

Example B: Test pass rate

If 90 of 100 executed test cases pass, the pass rate is 90%. The remaining 10 failures still need triage. The percentage does not describe severity, untested requirements, or whether the test cases are well designed.

Example C: Availability

If a service is available for 99 hours during a 100-hour observation window, measured availability is 99%, assuming the team's agreed definition of available time and exclusions.

> **Ethical use of metrics:** avoid ranking people by raw bug counts, lines of code, or story points. Such measures can encourage gaming and ignore complexity, collaboration, and risk.

7. Software Maintenance

Software maintenance is the modification of software after delivery to correct problems, adapt to changes, improve qualities, or prevent future issues. Maintenance can take a large share of a system's lifetime because software must evolve with users, rules, devices, dependencies, and operating environments.

Maintenance type	Purpose	CEMS example
Corrective	Fix a discovered fault	Fix duplicate registrations caused by a defect
Adaptive	Adapt to a changed environment or external rule	Update integration after the college changes its identity provider
Perfective	Improve performance, usability, or functionality	Add event filters requested by students
Preventive	Reduce future failure or maintenance risk	Refactor duplicated registration logic and add tests before further changes

These categories describe the purpose of a change. A single maintenance release may include more than one type.

8. A Safe Maintenance Workflow

1. Receive and clarify the change request or incident report.

2. Assess impact, urgency, security implications, dependencies, and affected requirements.

3. Create a traceable issue and agree on acceptance criteria.

4. Design the smallest appropriate change and review risks.

5. Implement the change in version control on a controlled branch or workflow.

6. Run relevant unit, integration, system, and regression tests.

7. Review and approve the change according to team policy.

8. Deploy using the agreed release process and monitor results.

9. Update documentation and close the issue with evidence.

> A quick production fix should still have suitable review, testing, traceability, and rollback planning. Urgency may change the workflow, but it does not make risk disappear.

9. Technical Debt and Maintainability

Technical debt is the future cost created by shortcuts or design choices that make later changes harder. Some shortcuts are deliberate and justified; unmanaged debt can accumulate and slow development or increase failure risk.

CEMS example: if registration rules are copied into several controllers, a new capacity rule may need multiple edits. One edit may be missed, causing inconsistent behavior. Moving the rule into a well-defined service and adding tests can reduce that risk.

• Prefer clear names and small, cohesive modules.

• Remove duplication where doing so improves clarity and reliability.

• Add automated tests around important business rules.

• Document decisions and non-obvious constraints.

• Use code review and refactoring to keep changes understandable.

10. Classroom Activity: Quality Dashboard

Imagine CEMS has just released a new registration feature. Use this sample data to discuss quality; it is illustrative, not a real production report.

Measure	Sample result	Question to ask
Executed tests	90 passed, 10 failed	Which failures block release and why?
Open defects	2 critical, 5 major, 8 minor	What risk remains?
Performance	95th percentile response time = 2.4 seconds	Does this meet the agreed target?
Availability	99% over the observation window	How much downtime occurred and what caused it?
Maintenance requests	6 requests after release	Are they defects, adaptations, or enhancements?

1. Choose three measures that matter most to students and event organizers.

2. Identify what extra information is needed before judging release quality.

3. Classify the six maintenance requests by purpose using realistic examples.

4. Recommend two process improvements to prevent recurring defects.

> **Deliverable:** a one-page quality dashboard interpretation with evidence, risks, and recommended actions.

11. Review Questions with Suggested Answers

1. What is software quality?

Suggested answer: The degree to which software meets requirements, standards, and intended user needs.

2. How does QA differ from testing?

Suggested answer: QA improves processes to prevent defects; testing evaluates behavior and helps reveal defects.

3. Give two examples of QA activities.

Suggested answer: Requirements reviews, coding guidelines, quality planning, process audits, or training.

4. What is a software metric?

Suggested answer: A quantitative measure related to a product, process, or project.

5. Calculate defect density for 12 defects in 6 KLOC.

Suggested answer: 12 ÷ 6 = 2 defects per KLOC, using the stated counting method.

6. Does high code coverage prove software quality?

Suggested answer: No. Tests may lack meaningful assertions and may not cover requirements or realistic scenarios.

7. Name the four common maintenance types.

Suggested answer: Corrective, adaptive, perfective, and preventive.

8. Give an adaptive maintenance example.

Suggested answer: Changing an integration because an external service or operating environment changed.

9. What is technical debt?

Suggested answer: Future cost or risk caused by choices that make later changes more difficult.

10. Why should metrics be interpreted in context?

Suggested answer: Definitions, severity, project characteristics, and collection methods affect what the numbers mean.

12. Glossary

Term	Meaning
Quality Assurance	Activities that improve processes and help prevent defects
Quality Control	Activities that evaluate work products against quality criteria
Metric	A quantitative measure used to understand a product, process, or project
Defect density	Defect count divided by a defined software size measure
Code coverage	A measure of code elements exercised by tests
Technical debt	Future cost or risk created by choices that make changes harder
Corrective maintenance	Modification to fix a discovered fault
Adaptive maintenance	Modification to suit a changed environment
Perfective maintenance	Modification to improve functionality or quality
Preventive maintenance	Modification intended to reduce future problems

13. Homework

1. Prepare a quality plan for a small college application. Include at least four quality goals, review points, and testing activities.

2. Calculate the pass rate for 84 passing tests out of 100 executed tests. Explain what the number does not tell you.

3. Classify five proposed changes to CEMS as corrective, adaptive, perfective, or preventive maintenance.

4. Choose three metrics for a student project and define exactly how each will be measured.

5. Describe one example of technical debt and a safe plan to reduce it.

Next chapter: Project Management Fundamentals, Planning, Work Breakdown Structure (WBS), and Scheduling.